

Problem Title

Find the Index of the First Occurrence in a String

Difficulty Level: Easy

Problem Description (Including Scenario)

You are given two strings:

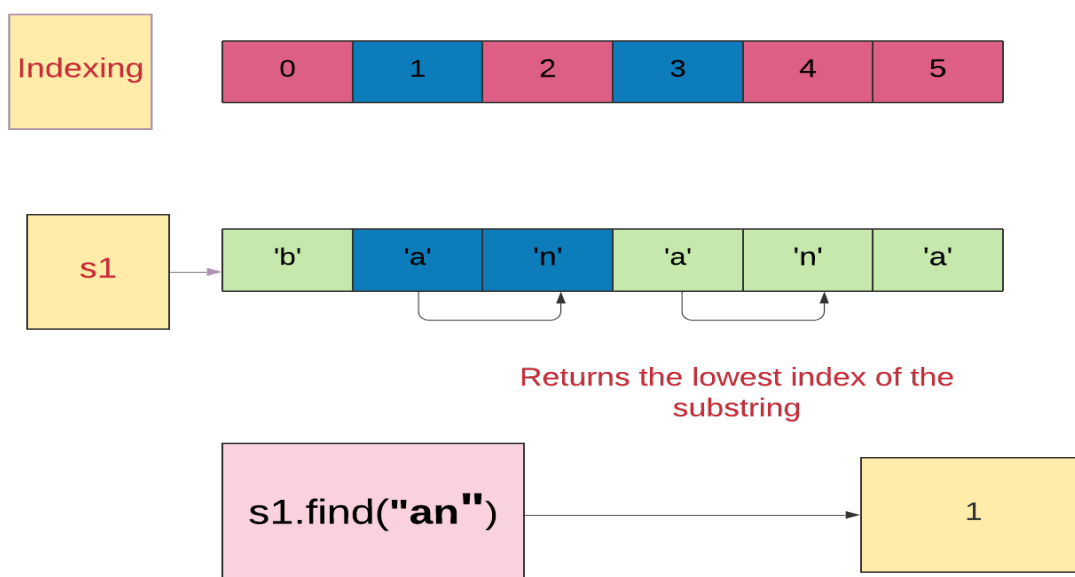
- **haystack** → the main string
- **needle** → the substring to search for

Your task is to find the **index of the first occurrence** of needle inside haystack.

- If needle exists in haystack, return the **starting index** of its **first occurrence**.
- If needle does **not** exist, return **-1**.

This problem is commonly used in **text searching**, **search engines**, and **string processing systems**.

Visualized Example (Explanation)



Text

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Character	a	b	c	a	b	a	a	b	c	a	b	a	c

Pattern

a	b	a	a
---	---	---	---

Example 1

haystack = "sadbutsad"

needle = "sad"

Step-by-step Visualization:

Index: 0 1 2 3 4 5 6 7 8

Haystack: s a d b u t s a d

Needle: s a d

- "sad" matches at index 0
- Another match exists at index 6
- First occurrence is at index 0

Output: 0

Example 2

haystack = "leetcode"

needle = "leeto"

- "leeto" is **not present** in "leetcode"

Output: -1

Input Format

- A string haystack
- A string needle

Output Format

- Return a **single integer**:
 - Index of first occurrence of needle
 - -1 if not found

Constraints

$1 \leq \text{haystack.length} \leq 10^4$

$1 \leq \text{needle.length} \leq 10^4$

Both strings contain only lowercase English letters

Topic Covered

- Strings
- Substring Search [Binary Search]
- Brute Force

Approaches

1. Brute Force Approach

Idea:

Check every possible starting index in haystack and compare characters of needle.

Algorithm:

1. Loop from index 0 to $\text{haystack.length} - \text{needle.length}$
2. For each index, compare characters of needle
3. If all characters match $\rightarrow$ return index
4. If no match found $\rightarrow$ return -1

Time Complexity: $O(n \times m)$

Space Complexity: $O(1)$

(n = haystack length, m = needle length)

2. Optimized Approach (Built-in / Sliding Window)

Idea:

Use efficient substring comparison or built-in methods (internally optimized).

- Java internally uses optimized string matching.
- Reduces unnecessary comparisons.

Time Complexity: $O(n)$ (average case)

Space Complexity: $O(1)$

Java Solution (Preferred Language)

Optimized Java Solution

```
class Solution {
    public int strStr(String haystack, String needle) {
        if (needle.length() == 0) return 0;

        for (int i = 0; i <= haystack.length() - needle.length(); i++) {
            if (haystack.substring(i, i + needle.length()).equals(needle)) {
                return i;
            }
        }
        return -1;
    }
}
```

Practice Link (Live)

- LeetCode
<https://leetcode.com/problems/find-the-index-of-the-first-occurrence-in-a-string/>
- GeeksforGeeks
<https://practice.geeksforgeeks.org/problems/implement-strstr/1>

Video Solution Link

- YouTube
<https://www.youtube.com/watch?v=joF0Z7nVSrA>

Learning Outcome

After solving this problem, the learner will be able to:

- Understand substring search logic
- Apply brute force and optimized strategies
- Work confidently with Java String APIs
- Analyze time and space complexity
- Solve real interview-level string problems